



Universitatea Tehnică „Gheorghe Asachi” din Iași
Facultatea de Automatică și Calculatoare



Sistem Local de Gestiune Criptografica si Analiza Comparativa a Performantei Algoritmilor AES si RSA in OpenSSL si Python

Proiect realizat de:

Avramescu Andrei

Popa Alexandru Serban

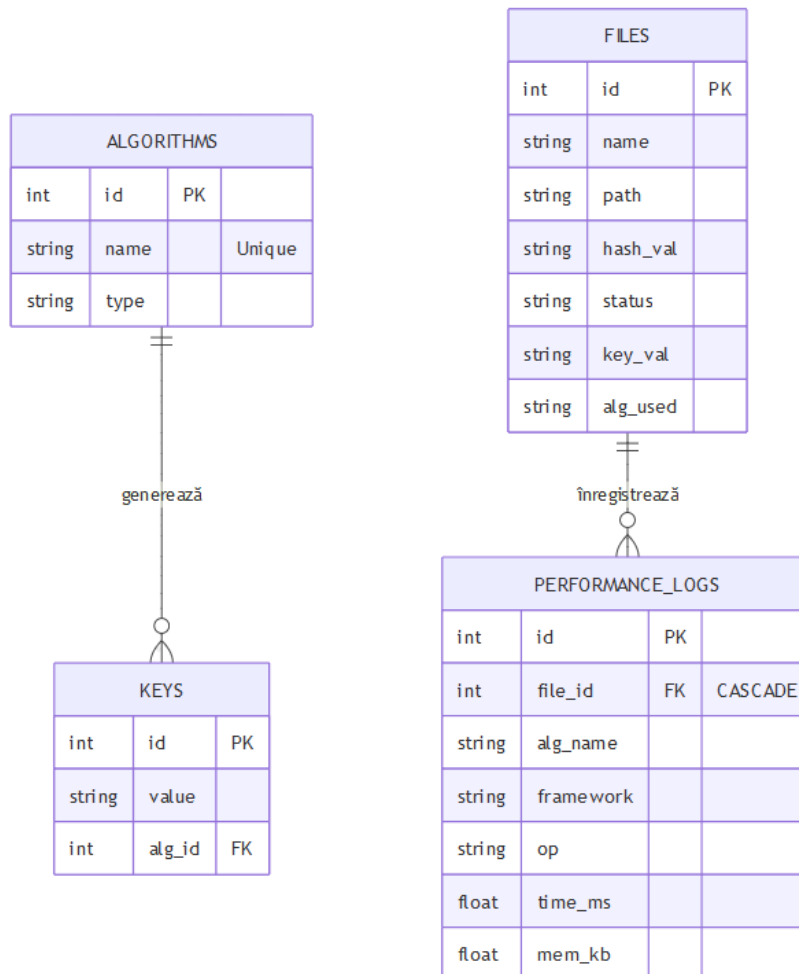
I. *Prezentare generala*

Proiectul realizat este o aplicatie desktop pentru criptarea si decriptarea fisierelor, dezvoltata in Python. Aplicatia ofera utilizatorului posibilitatea de a alege intre doi algoritmi de criptare, **AES-256** si **RSA-2048**, precum si intre doua moduri de implementare: folosind utilitarul extern **OpenSSL** sau biblioteca Python **cryptography**. Interfata grafica este realizata cu **CustomTkinter**, iar pentru persistenta datelor este utilizata o baza de date **SQLite**, accesata prin **SQLAlchemy**.

Rolul bazei de date este de a memora informatii despre:

- algoritmii disponibili;
- cheile utilizate;
- fisierele procesate;
- performanta operatiilor de criptare si decriptare.

II. *Diagrama Entitate-Relatie*



Entitatea Algorithm - Contine algoritmii disponibili in aplicatie și are attributele :

- **id** - cheie primara
- **name** - numele algoritmului
- **type** - tipul algoritmului (Symmetric / Asymmetric)

Entitatea KeyRecord - Contine cheile memorate in baza de date si are attributele :

- **id** - cheie primara
- **value** - valoarea cheii
- **alg_id** - cheie straina catre Algorithm.id

Entitatea FileRecord - Contine informatii despre fisierele procesate si are attributele :

- **id** - cheie primara
- **name** - numele fisierului
- **path** - calea fisierului
- **hash_val** - hash-ul SHA-256 al fisierului
- **status** - starea fisierului (Criptat / Decriptat)
- **key_val** - cheia utilizata
- **alg_used** - algoritmul utilizat

Entitatea PerformanceLog - Contine informatii despre performanta operatiilor efectuate si are attributele :

- **id** - cheie primara
- **file_id** - cheie straina catre FileRecord.id
- **alg_name** - numele algoritmului
- **framework** - framework-ul folosit (OpenSSL / Cryptography Lib)
- **op** - tipul operatiei (Enc / Dec)
- **time_ms** - timpul de executie in milisecunde
- **mem_kb** - memoria utilizata in KB

III. Explicatii privind implementarea

In cadrul proiectului au fost alesi doi algoritmi reprezentativi pentru doua categorii importante de criptografie:

AES-256

AES este un algoritm de criptare simetrica, ceea ce inseamna ca aceeași cheie este folosita atat pentru criptare, cat si pentru decriptare. In proiect, AES este utilizat in varianta **AES-256-CBC**, ceea ce ofera un nivel ridicat de securitate. Pentru implementarea in Python, cheia introdusa de utilizator este transformata intr-o cheie de 256 biti folosind functia SHA-256.

RSA-2048

RSA este un algoritm de criptare asimetrica, folosind o pereche de chei: una publica si una privata. In proiect este folosita o cheie de 2048 biti, dimensiune suficient de sigura pentru scopul aplicatiei.

Prin alegerea acestor doi algoritmi, proiectul acopera ambele mari categorii de criptare si permite observarea diferentelor dintre ele.

Pentru determinarea precisa a performantei in cazul framework-ului OpenSSL, aplicatia calculeaza consumul RAM ca diferenta a memoriei RSS (Resident Set Size) inainte si dupa operatie, iar timpul de executie efectiv este obtinut prin scaderea overhead-ului de lansare a procesului extern (determinat dinamic prin executia prealabila a comenzii 'openssl version'), eliminand astfel complet latenta sistematica indusa de schimburile de context din subprocess.

In cazul framework-ului nativ Python (Cryptography Lib), evaluarea performantei este directa si nu implica un proces extern, timpul de executie fiind determinat prin diferenta citirilor ceasului de inalta rezolutie 'time.perf_counter()', in timp ce consumul de memorie RAM este calculat in mod similar ca diferenta a memoriei fizice RSS (Resident Set Size) ocupate de aplicatie inainte si dupa rularea functiilor criptografice.

IV. Fluxul de functionare al aplicatiei

Fluxul general este urmatorul:

1. Utilizatorul porneste aplicatia.
2. Aplicatia incarca configurarea din options.ini.
3. Utilizatorul alege algoritmul si framework-ul.
4. Utilizatorul introduce sau selecteaza o cheie.

5. Utilizatorul alege un fisier.
6. Aplicatia apeleaza functia corespunzatoare din CryptoEngine.
7. Se genereaza fisierul criptat sau decriptat.
8. Rezultatul este inregistrat in baza de date.
9. Se salveaza timpul si memoria utilizata.
10. Interfata se actualizeaza cu noile date.

V. Concluzii

Prin acest proiect a fost realizata o aplicatie desktop pentru criptarea si decriptarea fisierelor, avand la baza o baza de date relationala pentru stocarea informatiilor importante. Aplicatia permite utilizarea algoritmilor AES-256 si RSA-2048, atat prin biblioteca `cryptography`, cat si prin OpenSSL, oferind astfel posibilitatea de a compara doua metode diferite de implementare.

Baza de date are un rol important in proiect, deoarece pastreaza informatii despre fisierele procesate, algoritmi disponibili, cheile utilizate si rezultatele masuratorilor de performanta. Astfel, aplicatia nu executa doar operatii de criptare si decriptare, ci ofera si un istoric al activitatii realizate.

Un avantaj al proiectului este structura modulara, fiecare componenta avand un rol clar: interfata grafica, motorul criptografic si gestionarea bazei de date. Aceasta organizare face aplicatia mai usor de inteles, testat si extins.